# STATISTICAL COMPUTATION & SIMULATION
# HOMEWORK 2

Assigned February 16, 2026, due March 19, 2026

*This homework pertains to materials covered in the second month of the semester. Your final report need to be typed, with your team member names listed on the document, and with properly labeled computer output for those problems that require computer simulations. To obtain full credit, I suggest you embed your (`R`, `python`, or whatever you are using) input code within the main text and organize them well so they are easy to read. You also need to attach your code on Canvas as a separate file and clearly indicate the problem it corresponds to. Please also describe your work and findings with sufficient details. Upload the file to the Homework 2 on Canvas before class on March 19.*

*While AI tools (e.g., ChatGPT) might help you explore ideas, they are not a substitute for deriving results yourself. Submissions that rely on AI outputs or on solutions from previous years will receive a zero, and such cases will be reported to the Office of Academic Integrity.*

**Problem 1** (Numerical integration; 40 points)**.** Choose `R` or `Python`. Your task is to implement the *lower* and *upper* incomplete gamma functions,

$$\gamma(s,x) = \int_0^x t^{s-1}e^{-t}\,dt, \qquad \Gamma(s,x) = \int_x^\infty t^{s-1}e^{-t}\,dt,$$

as defined on this Wikipedia page. Both are improper integrals (at 0 and/or $\infty$), so numerical integration must be handled carefully.

(1) Define the integrand in `R`/`Python` and perform the numerical integration in `R`/`Python`.
(2) Define the integrand in `C++`, call it from `R`/`Python`, and perform the numerical integration in `R`/`Python`.
(3) (Not required; successful implementation gets 10 bonus points) Implement both the integrand *and* the numerical integration in `C++`, then import the `C++` function in `R`/`Python`.

**Task 1.** Evaluate each of your implementations on the following grid of parameter values:

$$s \in \texttt{seq}(0.0001, 10, \texttt{length} = 100), \qquad x \in \texttt{seq}(0.0001, 10000, \texttt{length} = 100),$$

which yields $100 \times 100 = 10{,}000$ $(s,x)$ pairs. Report the wall-clock time required to compute:

- the lower incomplete gamma $\gamma(s,x)$ over all 10,000 pairs, and
- the upper incomplete gamma $\Gamma(s,x)$ over all 10,000 pairs,

for each method above. Your functions should accept vector inputs (or otherwise be evaluated efficiently over the grid). The team with the shortest running time will receive an additional 20 bonus points; note that I will run your codes on my computer to ensure a fair comparison and to validate your integration to see if they match with the baseline functions (see Task 2).

**Task 2.** Using the *same* grid, compare the runtime of your implementations to a trusted library baseline:

- In `R`:

  ```
  pgamma(x, shape = s, scale = 1, lower.tail = TRUE) *
                                gamma(s)
  ```

  for $\gamma(s, x)$, and

  ```
  pgamma(x, shape = s, scale = 1, lower.tail = FALSE) *
                                gamma(s)
  ```

  for $\Gamma(s, x)$.
- In `Python`:

  ```
  scipy.special.gammainc(s, x) * scipy.special.gamma(s)
  ```

  for $\gamma(s, x)$, and

  ```
  scipy.special.gammaincc(s, x) * scipy.special.gamma(s)
  ```

  for $\Gamma(s, x)$.

For each method and for both $\gamma(s, x)$ and $\Gamma(s, x)$, compute the mean squared error (MSE) between your outputs and the corresponding library baseline over the full grid of 10,000 $(s, x)$ pairs, and report these MSEs to verify numerical correctness.

The following is an example code snippet for the time experiment:

```python
# ------------------------- In python ---------------------------
import numpy as np
import pandas as pd
from time import perf_counter

s_vals = np.linspace(0.0001, 10.0, 100)
x_vals = np.linspace(0.0001, 10000.0, 100)
param_set = pd.MultiIndex.from_product([s_vals, x_vals],
    names=["s", "x"]).to_frame(index=False)

# Assume these are your vectorized functions:
# lower_incomplete_gamma(s_array, x_array)
# upper_incomplete_gamma(s_array, x_array)

t0 = perf_counter()
lower_incomplete_gamma(param_set["s"].to_numpy(),
    param_set["x"].to_numpy())
t1 = perf_counter()
```

```
t1 - t0

t0 = perf_counter()
upper_incomplete_gamma(param_set["s"].to_numpy(),
    param_set["x"].to_numpy())
t1 = perf_counter()
t1 - t0

# Or
# ------------------------- In R ---------------------------
s_vals <- seq(0.0001,10,length.out=100)
x_vals <- seq(0.0001,10000,length.out=100)
param_set <- expand.grid(s=s_vals, v=v_vals)

# Assume lower_incomplete_gamma() and upper_incomplete_gamma()
#   are the functions you implemented
ptm <- proc.time()
lower_incomplete_gamma(param_set$s, param_set$x)
proc.time() - ptm

ptm <- proc.time()
upper_incomplete_gamma(param_set$s, param_set$x)
proc.time() - ptm
```

---

**Problem 2** (2-dim numerical integration; 40 points)**.** Consider the following survival function

$$1 - F^*(x) = \sqrt{\frac{1}{\pi}}\gamma\left(\frac{1}{2}, \frac{\tau}{2x^{1/\phi}}\right) + x^{-1}\sqrt{\frac{1}{\pi}}\left(\frac{\tau}{2}\right)^{\phi}\Gamma\left(\frac{1}{2} - \phi, \frac{\tau}{2x^{1/\phi}}\right).$$

where $\phi > 0$, $\tau > 0$, $\gamma(\cdot,\cdot)$ and $\Gamma(\cdot,\cdot)$ are the lower and upper incomplete Gamma functions, respectively.

**Task 1.** Using your incomplete-gamma implementations from Problem 1, numerically verify that for a fixed pair $(\phi, \tau) = (0.7, 2)$, $F^*(x)$ defines a valid CDF on $(0, \infty)$ by checking:

(1) $0 \le F^*(x) \le 1$ for all $x$ on a sufficiently fine grid;
(2) $F^*(x)$ is non-decreasing in $x$; and
(3) $\lim_{x\downarrow 0} F^*(x) = 0$ and $\lim_{x\to\infty} F^*(x) = 1$.

(Note that the upper incomplete Gamma function is well-defined for negative $s$ too. You should check and see if your upper function is valid for negative $s$ values.)

Now suppose $X \sim F^*$, $\epsilon \sim N(0, \sigma^2)$, and $X \perp \epsilon$. Define $Y = X + \epsilon$, i.e., $Y$ is obtained by perturbing $X$ with additive Gaussian (white) noise. The

distribution function of $Y$ admits the following convolution representation:

$$\begin{aligned} F_Y(y) = \Pr(Y \le y) &= \int_0^\infty \Phi\left(\frac{y-x}{\sigma}\right) dF^*(x) \\ &= \int_0^\infty F^*(x)\,\frac{1}{\sigma}\,\varphi\left(\frac{y-x}{\sigma}\right) dx, \end{aligned} \tag{1}$$

where $\Phi(\cdot)$ and $\varphi(\cdot)$ denote the standard normal CDF and PDF, respectively. The second equality follows from integration by parts applied to the integral in the first line, using $F^*(0) = 0$ and $\Phi((y-x)/\sigma) \to 0$ as $x \to \infty$.

The integral in (1) depends on three parameters $(\phi, \tau, \sigma)$. Moreover, evaluating the integrand $F^*(x)\,\varphi((y-x)/\sigma)/\sigma$ is itself nontrivial because $F^*(x)$ is defined through incomplete gamma functions. Thus, numerical evaluation of $F_Y(y)$ amounts to a *nested* improper integration problem, requiring stable and efficient computation of incomplete gamma terms across a range of $(\phi, \tau, \sigma)$ and $y$.

**Task 2.** In this task you will numerically evaluate the convolution integral in (1) to compute and visualize the CDF $y \mapsto F_Y(y)$ of $Y = X + \epsilon$. Consider the two parameter settings

$$(\phi, \tau, \sigma) = (0.3, 2, 3) \quad \text{and} \quad (\phi, \tau, \sigma) = (0.7, 2, 3),$$

and reproduce the CDF curves shown in Figure 1. Specifically, compute $F_Y(y)$ on two dense grids:

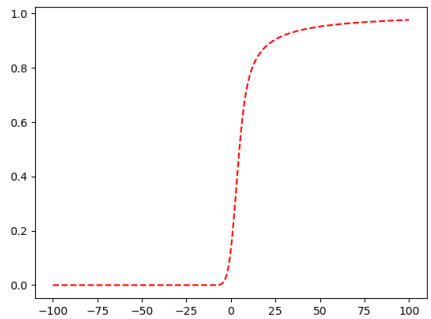
$$y \in [-100, 100] \quad \text{and} \quad y \in [10^3, 10^4],$$

each with 2000 equally spaced points:

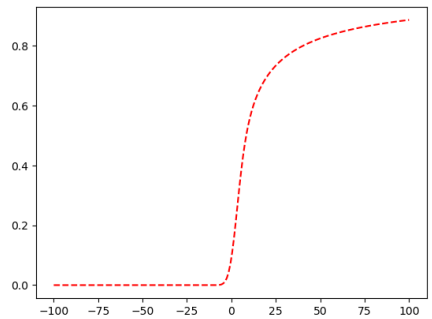
```python
# In python
import numpy as np
y_vals1 = np.linspace(-100,100,num=2000)
y_vals2 = np.linspace(1000,10000,num=2000)

# Or
# In R
y_vals1 <- seq(-100,100,length.out=2000)
y_vals2 <- seq(1000,10000,length.out=2000)
```

(1) **CDF plots.** Reproduce Figure 1 in either `R` or `Python` (four panels total: two parameter settings $\times$ two $y$-ranges). Your plotted CDF values should be smooth and monotonically non-decreasing in $y$, and the results should be consistent across any alternative implementations you attempt.

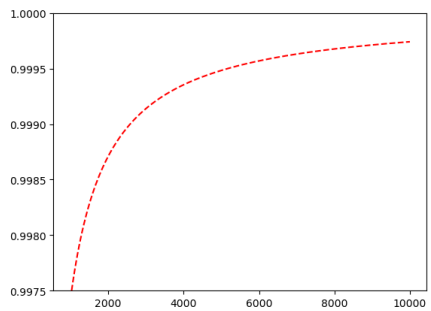
(2) **Runtime comparison.** For each $\phi \in \{0.3, 0.7\}$, report the wall-clock time required to evaluate $F_Y(y)$ at all 4000 grid points (2000 in



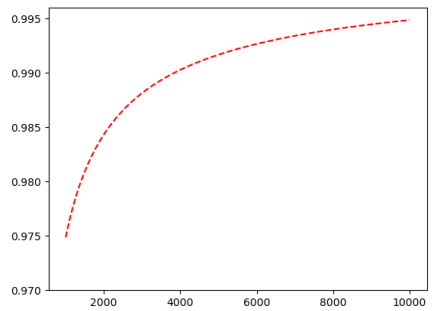

(A) $(\phi, \tau, \sigma) = (0.3, 2, 3)$, $y \in [-100, 100]$




(B) $(\phi, \tau, \sigma) = (0.7, 2, 3)$, $y \in [-100, 100]$




(C) $(\phi, \tau, \sigma) = (0.3, 2, 3)$, $y \in [10^3, 10^4]$



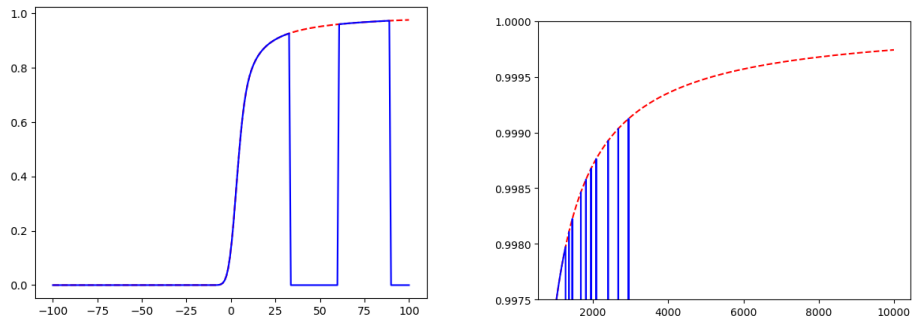

(D) $(\phi, \tau, \sigma) = (0.7, 2, 3)$, $y \in [10^3, 10^4]$

FIGURE 1. Visualization of the CDF $F_Y(y)$ for $y \in [-100, 100]$ (top panels) and $y \in [10^3, 10^4]$ (bottom panels) under two different hyperparameter settings. We can see that both distributions are very heavy-tailed, but $\phi = 0.7$ gives a much heavier tail.

$[-100, 100]$ and 2000 in $[10^3, 10^4]$). If you implement multiple numerical strategies (e.g., different quadrature rules, variable transformations, or C++ acceleration), report runtimes for each and comment briefly on stability and accuracy.

This integral is intentionally challenging: naive adaptive quadrature can fail for large $y$, producing spurious "leakage" in which the computed CDF incorrectly drops toward 0 rather than approaching 1; see Figure 2. Both `scipy.integrate` and `GSL` call `QUADPACK`-style adaptive routines, which may exhibit this pathology in this setting. You are encouraged to mitigate these issues using numerically stable strategies (e.g., a change of variables, truncation based on Gaussian mass, splitting the integration domain, or computing the survival function when $F_Y(y)$ is near 1).

The team with the shortest running time will receive an additional 20 bonus points; note I will run your code on my machine to verify correctness and to ensure a fair runtime comparison. Please provide a single script/notebook and include any required compilation instructions (if applicable).




(A) $(\phi, \tau, \sigma) = (0.3, 2, 3)$, $y \in [-100, 100]$ (B) $(\phi, \tau, \sigma) = (0.3, 2, 3)$, $y \in [10^3, 10^4]$

FIGURE 2. Visualization of the failed evaluations of $F_Y(y)$ using `QUADPACK` in blue compared to the true $F_Y(y)$ values in red. Instead of being a smooth function of $x$, the adaptive quadrature in `QUADPACK` results in "leakages" of the CDF values, i.e., the numerical CDF value is close to 0 instead of being close to 1 for the large $x$ value.

**Problem 3** (Sampling from a distribution; 30 points)**.** The exponentially tilted positive stable distribution is very useful and flexible in modeling extreme weather. One family of such distributions is parameterized by $\theta > 0$. Its density function can be written as:

$$f(x;\theta) = \sqrt{\frac{1}{4\pi}} \frac{\exp(-\frac{1}{4x} - \theta x)}{x^{3/2}\exp(-\sqrt{\theta})}, \ x > 0. \tag{2}$$

You can verify the two necessary and sufficient conditions for $f$ to be a valid density function.

(1) Use any sampling techniques covered in class (e.g., rejection sampling, importance sampling, etc.) that you prefer, and sample $N = 100,000$ independent samples for $\theta = 0.1$;
(2) Replicate Figure 3; that is, plot the sample density using histogram and compare it to the underlying true density (2). What we expect is almost perfect alignment between the sample density and the analytic density. The code needed to generate Figure 3 is attached here:

```
hist(samples, breaks = 1000, freq = FALSE, border =
    'lightblue')
curve(true_density, from = 1e-5, to = 60, n=1000, add =
    TRUE, col='red', lty=3)
```

Here, `samples` is the vector that stores the $N$ samples, and `true_density()` is an `R` function you can easily write to calculate the true density (2).

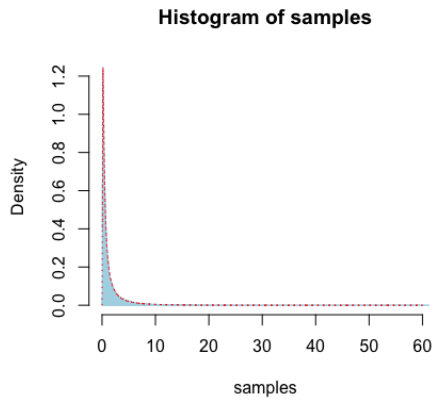



FIGURE 3. The density histogram of the $N = 100,000$ independent samples from the density (2) with $\theta = 0.1$ (in lightblue), with the true analytic density superimposed on top (in red dashline).

**Problem 4** (Markov chain Monte Carlo; 60 points)**.** Consider a $4 \times 4$ grid of Bernoulli random variables denoted by $X = [X_{ij}]$. Note that this matrix can be thought of as a small $4 \times 4$ black and white (binary) image. Now let $N(X)$ be the total number of successes in $X$ (the sum of $X$) and $D(X)$ be the total number of neighbors (horizontal or vertical) which differ. For example, if

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \; N(X) = 1, \; D(X) = 2,$$

or if

$$X = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \; N(X) = 3, \; D(X) = 5.$$

Note that if you write the elements of $X$ out in a vector $V$, then there is a $24 \times 16$ matrix $M$ such that $D(X)$ is sum of the absolute values of $MV$ . The 24 arises because there are 24 neighbor-pairs to check (diagonal elements are not neighbors.)

Suppose that the probability distribution of X is proportional to

$$\pi(X) \propto p^{N(X)}(1-p)^{16-N(X)} \exp\{-\lambda D(X)\}.$$

Therefore, if $\lambda = 0$, we have i.i.d. Bernoulli entries.

Use Markov chain Monte Carlo to approximate the probability that the diagonal of this matrix is all 1's. Assume $p = 0.8$. Approximate the probabilities for each of the following cases: $\lambda = 0.5$, $\lambda = 1$.

Consider two choices of proposals for the Metropolis-Hastings algorithm: $Q_1$ where every transition is equally likely with probability $1/2^{16}$. That is, the next candidate state is simply 16 i.i.d. Bernoulli variables with probability 0.5 of success each. Let $Q_2$ be such that you choose one of the 16 entries in the table with probability 1/16 and then set that entry to 0 or 1 with probability 0.5 each. Thus at most one entry in $X$ can change at each step using $Q_2$.

Report the following (10 points for each item):

(1) A brief but clear description of the algorithm you used. You should also provide important details such as starting values (e.g. arbitrary values, values based on several preliminary MCMC runs, a random draw from some initial distribution you chose etc.), proposals for your algorithm (different ways in which you constructed the updates.)
(2) Which of the algorithms you tried resulted in a faster mixing Markov chain? Provide some justification for your answer (e.g., refer to specific plots, compare effective sample sizes for the same Markov chain lengths, compare MCMC standard errors for the same Markov chain length.)
(3) Relevant labelled plots along with clear descriptions of the plots. Since you are running several chains (two proposals $Q_1$, $Q_2$ and different values of $\lambda$), you may not be able to share all your plots. Pick the most representative/useful ones. Write down the length of the chain on which you based your final estimates. Explain how you used the plots and any other information to make your decisions (on how long to run the chain, whether your chain was mixing reasonably well, why one algorithm was better than another etc.) Some examples: plots of autocorrelations, MCMC standard error estimates, estimate versus sample size.
(4) In a table report all estimates, along with associated MCMC standard errors (use consistent batch means which is implemented in the package `batchmeans`; see this weblink).
(5) Briefly state the main theoretical results (as discussed in class) that justify the use of the Markov chains you constructed above for estimating expectations with respect to $\pi$. For full credit, state this in detail.
(6) Construct a more efficient Metropolis-Hastings algorithm than the ones you have used above. Do this for just one case, $\lambda = 1$. Justify that your new algorithm is more efficient than the others (e.g., by e comparing effective sample sizes or MCMC standard errors for the same run lengths, or acf plots.) If you are unable to obtain a more

efficient algorithm, for partial credit simply report your results from a different algorithm from the ones constructed above.

**Problem 5** (Bayesian inference; 50 points)**.** Fit the following nonlinear regression model:

$$Y_i = \frac{1 + \beta_1 X_i}{1 + \beta_2 \exp(\beta_3 X_i)} + \epsilon_i,\ i = 1, \ldots, n,$$

where $\epsilon_1, \ldots, \epsilon_n \overset{\text{i.i.d.}}{\sim} N(0, \sigma^2)$. Assume the following prior distributions:

$$\beta_1 \sim N(0, \tau_1^2 = 10),\ \beta_2 \sim N(0, \tau_2^2 = 10),\ \beta_3 \sim N(0, \tau_3^2 = 10),$$

$$\sigma^2 \sim \text{Gamma}(k = 1, \theta = 10) \text{ under the shape-scale parameterization.}$$

Infer $\beta_1$, $\beta_2$, $\beta_3$ and $\sigma^2$ based on the joint distribution. The data you will use is the `glm_dat.RData` on Canvas. Report the following:

(1) (6 points) Provide pseudocode for your final MCMC algorithm.
(2) (8 points) Provide diagnostics to show that your MCMC algorithm-based approximations are reliable.
(3) (8 points) Summarize approximations of the posterior expectations for each of the 4 variables. Also show the MCMC standard errors for each approximation.
(4) (8 points) Plot posterior PDFs of each variable.
(5) (8 points) Does there appear to be strong dependence between any pair of random variables? Report any posterior cross-covariances and bivariate PDFs that suggest that it may not be easy to separate inference for some of the variables (that there may be identifiability issues).
(6) (10 points) Sensitivity analysis: Use importance sampling to approximate the posterior expectations for part (3) when the prior distribution for the $\beta$'s have variances of 5 each instead of 10.
(7) (10 points) Repeat the above using importance sampling for when variances are 50 each instead of 10. Do you have any concerns about your approximations?